Recovering Missing Trial Labels From Randomization Constraints: A Validation Pipeline for Small-Sample EEG Decoding

Bhoomika R Hegde

***Abstract*—Small EEG datasets often have many epochs but few subjects, and shared datasets often lack the codebook that links trigger codes to stimuli. A decoding score can then mislead in either direction. I describe a validation pipeline that tests on unseen subjects against within-subject baselines, audits data and events, and adds two stages. The first recovers labels from the experimental design: published randomization rules are written as constraints on the unknown code-to-label mapping and solved exactly, which scores the labels in use, counts the consistent mappings, and returns the mapping when it is unique. Recovered labels are fixed before the EEG is used and tested against other groupings of the same codes. The second adds simulated effects of known size, including effects that differ between subjects, to state what a negative result could have found. In simulation, the rule recovered the two-group (speaker) assignment exactly in all 100 runs with four distinct trial orders; the three-group (condition) assignment needed an extra assumption, which four orders supported only weakly and eight orders confirmed or rejected in all 40 runs. In a public canine EEG dataset (16 dogs, 2820 epochs), decoding with the alphabetically rebuilt labels was at chance (macro AUC 0.503, where 0.5 is chance). Their speaker assignment broke the published rule as often as random groupings do. The rule identified the speaker split exactly (odd versus even codes), and, assuming paired codes share a condition, a condition grouping that decoded above random and rule-consistent code groupings (0.537; $p \leq 0.013$), with one group differing from the other two at 200–300 ms, resembling the original report. The recovered labels are consistent with the design but not confirmed, and no claim is made about canine cognition.**

***Index Terms*—Electroencephalography, decoding, validation, label recovery, constraint programming, sensitivity analysis, canine EEG.**

I. Introduction

A classifier trained on EEG can score well for the wrong reasons. It can pick up which subject the signal came from, a recording artifact, the order of trials in a session, or a leak from preprocessing. It can also score badly for the wrong reasons: the test may be too weak to see a real effect, or the labels may simply be wrong. Both problems are worst when there are many epochs (short slices of signal) but few subjects, because thousands of epochs make a dataset look large while the number of independent subjects stays small.

Labels are the least checked part of this chain. In reused datasets, the link between the numeric trigger codes in the recordings and the stimuli they stood for often lives in a presentation script or codebook that was never released. Analysts then rebuild it, for example by matching codes to sound files in alphabetical order. If that guess is wrong, every downstream score tests the guess, and a null result says nothing about the EEG.

This paper describes a validation pipeline for small-sample EEG decoding that treats the subject as the unit, checks the data and events before any score is read, and adds two stages that are usually missing. The first asks whether the labels can be trusted, and if they cannot, whether the experimental design itself constrains them enough to recover them. The second asks how large an effect the test could have found. I show the pipeline on a public canine EEG dataset [1] of 17 recordings, 16 of which enter the analysis.

My contributions are:

1) *A staged validation pipeline* (Section III) in which every stage ends in PASS, WARN, or FAIL and limits what may be claimed. Decoding is always tested on unseen subjects against baselines built within each subject.
2) *Recovering labels from the experimental design.* When the file that says which trigger code means which condition is missing, the rules used to order the trials can still reveal it. For example, if a study states that no condition was ever played more than three times in a row, any guessed labelling that produces four in a row must be wrong. I write such rules as constraints on the unknown labelling of the 48 codes and use a solver to find every labelling that fits the recorded trial orders. This shows whether the labels in use are possible, and, if only one labelling fits, recovers it. The recovered labels are then checked once on the EEG, and I test the method on simulated experiments where the true labels are known.
3) *Checking what a negative result could have found.* I add artificial effects of known size to the real recordings, including effects that differ in timing, size, or direction between subjects, and measure how large an effect must be before the decoder finds it. A null result can then state the smallest effect it would have detected.

In the case study, the decoder scores at chance under the alphabetical labels used so far (macro AUC 0.503). The same recordings show that these labels break the published design rule far more often than a correct mapping could. Solving the rule recovers which codes share a speaker with no further assumption, and, under one stated assumption, which condition. With the recovered labels the same kind of decoder scores above baselines built from other code groupings, and the pattern of the effect, one group of words against the

other two, about 200–300 ms after the sound, resembles the original study's report. The recovered labels are consistent with the design but not confirmed by the data providers, and I report them as such. Sixteen dogs cannot support claims about canine cognition, and I make none. The question is whether the pipeline reports correctly what the data can and cannot support [2], [3].

II. Related Work

A. Canine EEG and Canine Cognition

Canine EEG studies have looked at sleep [4], visual responses [5], and auditory and social processing. They show that non-invasive recording works in awake dogs and that time-locked responses can be measured with careful task design. I use the word-processing dataset of Magyari et al. [1], in which dogs heard repeated recordings of known words, similar-sounding nonsense words, and dissimilar nonsense words. Kujala et al. [6], [7] used time-resolved classification on dog EEG recorded while dogs viewed faces and objects. Brain imaging has shown word processing in dogs [8], building on methods for scanning awake dogs [9].

B. Decoding, Leakage, and Testing on New Subjects

A decoding score does not say what the model learned. In small animal datasets a classifier can use dog identity, recording conditions, trial order, simple differences between stimuli, movement, or labeling errors. The wider EEG literature has shown how leakage and circular analysis [10] inflate scores through feature selection, normalization, model tuning, or cross-validation that mixes subjects. Del Pup et al. [11] showed that the way EEG data are split into training and test sets strongly affects cross-subject deep learning results. Validation must follow the intended use, which for a new subject means subject-wise splits [12], and leakage of this kind has inflated results across machine-learning-based science [13] and in translational EEG [14]. Subject-specific amplitude, anatomy, and movement can all become shortcuts. If the question is about a new dog, the test set must contain only dogs the model has not seen. I therefore use leave-one-dog-out testing.

C. Pseudoreplication, Shuffled Baselines, and Small Samples

Pseudoreplication means treating repeated measurements as if they were independent [15], [16]. It is a serious risk in canine EEG, where thousands of epochs may come from a dozen dogs. Shuffling labels gives a baseline for chance performance without strong assumptions [17], [18], but the shuffle must respect the data: shuffling across all epochs breaks dog membership and gives an irrelevant comparison. I shuffle within each dog and resample dogs, not epochs [19]. Cross-validated scores are very variable in small samples [20], [21], low power makes significant findings less likely to be true and exaggerates their size [2], and small studies with flexible analyses often fail to replicate [3]. Theoretical chance levels are also unreliable with few samples, so chance should be estimated empirically [22]. I respond by using the shuffled baseline, dog-level uncertainty, and open reporting of every warning.

D. Preprocessing, Quality Control, and Reproducibility

Filter and artifact settings change results. Human EEG pipelines stress early quality checks, reference handling, and documented artifact treatment, and tools such as FASTER and Autoreject choose thresholds systematically [23], [24]. Those thresholds still need a stated origin. Dogs add problems of their own: awake animals move their eyes, ears, head, and body, and no canine quality standard exists. The pipeline therefore records where each threshold came from. The FAIR principles [25] and BIDS [26], [27] describe how to organize data and metadata for reuse. I do not propose a canine EEG standard. I also record what each claim depends on: a missing stimulus codebook, an untracked preprocessing step, or uncommitted code changes all limit what can be said, so the pipeline records them as part of the result.

E. Missing Stimulus Metadata

Shared datasets often lose the file that links trigger codes to stimuli, and BIDS-style event files only help if that link was recorded in the first place [27]. When it is missing, analysts rebuild it from file names, ordering conventions, or memory, and the rebuilt labels are rarely tested. Randomization rules are a source of information that is usually ignored here: a pseudo-random order that never repeats a condition more than three times leaves a trace in every recording. Checking such rules is standard when a sequence is generated; I use them in reverse, as constraints on the unknown mapping, and solve them with a constraint-programming solver [28]. I did not find earlier work that recovers trigger-code mappings in this way.

To my knowledge, no canine EEG study has combined these points in one auditable pipeline that reports its own checking status as part of the result. I do not propose a new decoder.

III. The Validation Pipeline

The pipeline has six stages. PASS supports only the claim that was checked, WARN lets analysis continue with a stated caveat, and FAIL blocks the affected claim until new evidence resolves it. Grades never turn automatically into inclusion decisions, and a decoding score is not read until the stages before it have been reported.

A. Data and Events

A dataset specification records file layout, channel roles, sampling rate, the expected events, and where each assumption came from (verified, inferred, candidate, or unknown). Raw-data checks confirm that each recording loads and matches the specification. Event checks cover code range, missing, repeated and unknown codes, ordering, stimulus count, timing, and any published sequence rules; recordings that match the design exactly are listed apart from those that need review. Quality control measures duration, amplitude, variance, peak-to-peak values, non-finite samples, and event counts, records where each threshold came from, and removes no data. Confound checks report links between features and subject identity, session position, and pre-stimulus amplitude.

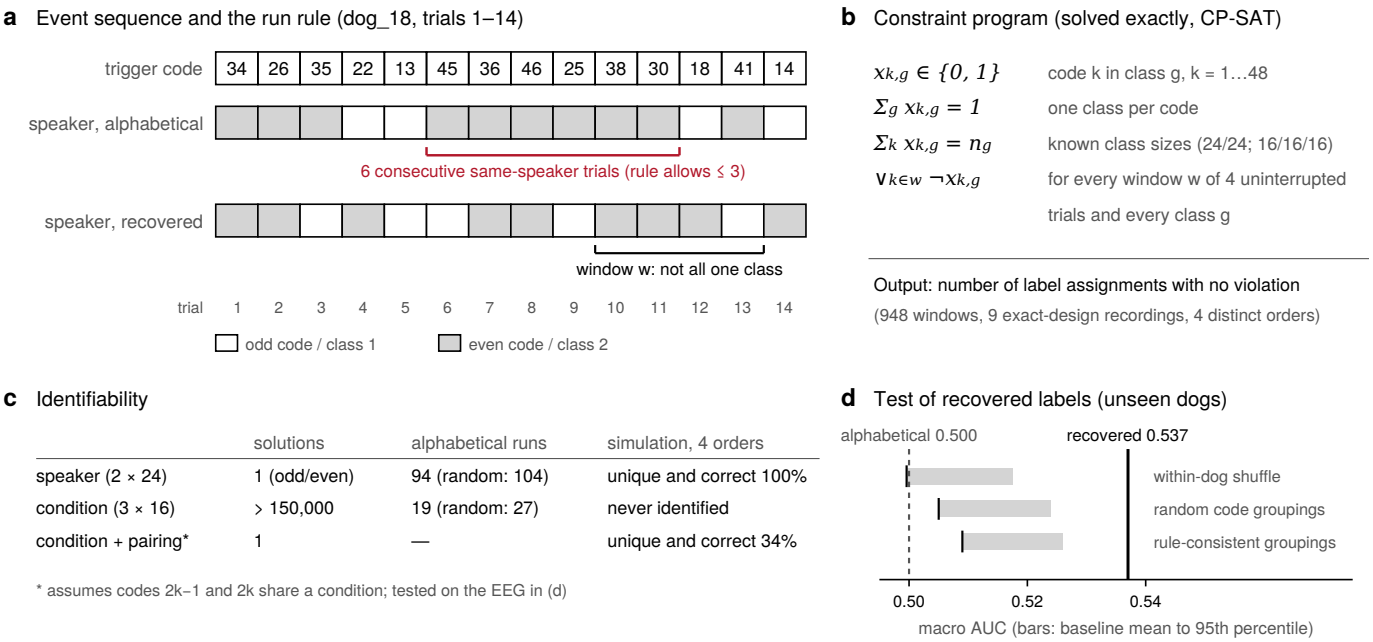


Fig. 1. Design-constrained label recovery (stage C), shown on the case study. (a) The first 14 trigger codes of `dog_18` with two speaker assignments. The alphabetical assignment produces six consecutive trials from one speaker, which the published rule forbids; the recovered assignment (odd versus even codes) never exceeds three. (b) The constraint program: one binary variable per code and class, fixed class sizes, and one clause per window of four uninterrupted trials and class. (c) Number of assignments without a violation, runs of four under the alphabetical labels (mean over 2000 random groupings in brackets), and recovery rates in simulations with four distinct trial orders. (d) Three-class decoding of the recovered condition groups on unseen dogs against three baselines (bars: mean to 95th percentile).

B. Label Origin

Checking that codes are intact is a different task from checking what they meant. The pipeline keeps the two apart: codes can be counted, ordered, and timed even when nobody knows which stimulus each one stood for. The origin of every label is recorded. Labels from a released codebook or presentation log are *verified*; labels rebuilt by the analyst are *candidate*. Candidate labels may be used to test the pipeline but not to make claims about the brain.

C. Design-Constrained Label Recovery

When labels are only candidates, the design often still says something about them. Fig. 1 shows the procedure on the case study. Let the experiment use K codes, each belonging to one class of an attribute (for example condition or speaker), with known class sizes. The recordings give event sequences $s^{(1)}, \ldots, s^{(R)}$. A rule of the form "no more than r consecutive trials share a class" says that for every window of $r + 1$ consecutive, uninterrupted events, the codes in the window must not all belong to the same class. I write the unknown assignment as binary variables $x_{k,g}$ (code k is in class g), with one class per code, the known class sizes, and, for every window w and class g, the clause

$$\bigvee_{k \in w} \neg x_{k,g}, \tag{1}$$

that is, at least one code in the window is not in class g. Windows are taken only from recordings that match the design exactly, so that a missing or extra event cannot create a false window. I solve the problem with a constraint-programming solver [28], break the symmetry of class names (class labels are arbitrary up to permutation), and enumerate the solutions (up to a cap when there are very many).

The output has three parts. First, the number of violations the labels in use produce, compared with random groupings of the same sizes; a correct mapping produces none. Second, the number of mappings with no violation: one means the attribute is identified, many means it is not. Third, any structural assumption needed to narrow the set (for example, that two codes are the same word read by two speakers) is listed, tested for feasibility, and compared with random versions of the same assumption, so that its effect can be judged. Class names are not identified by this step: the solver groups codes but does not name the groups.

Recovered labels are then frozen and tested once on the EEG (stage D). Because any grouping of codes can be decoded slightly above a within-subject shuffle when individual codes carry some signal of their own, the main comparison for recovered labels is a *code-level baseline*: the same decoder applied to many other groupings of the same codes, both uniformly random ones and ones that also satisfy the design rules.

D. Decoding on Unseen Subjects

All decoding is tested on subjects the model has not seen (leave-one-subject-out), with scaling and fitting inside the training folds only. Each score is compared with a baseline built by repeating the whole procedure with the labels shuffled within each subject, which keeps the number of epochs and

classes per subject. Code-level shuffles, circular shifts of the label sequence, and, for recovered labels, the code-level baseline above are added where they answer a different question. Uncertainty comes from resampling subjects. When several analysis versions are tried, the best score is compared with the best score in each shuffle (a family-wise test), not with a single-version baseline.

E. Sensitivity Calibration

A negative result is only informative if the test could have found something. I add a simulated difference between two conditions to the real recordings, scaled in multiples of the subjects' own average response to a stimulus, and record how often the full decoding procedure finds it against a baseline built on the same data. Besides an effect with the same shape and size in every subject, I simulate effects whose latency shifts between subjects, whose size varies, and whose sign is reversed in a quarter of subjects, because leave-one-subject-out testing can only find effects that generalize across subjects.

F. Claim Audit and Reproducibility

Every statement in the manuscript is classified as supported, partly supported, exploratory, or unsupported and linked to the report that supports it. Runs record input checksums, configuration, software versions, random seed, and command.

IV. Checking Label Recovery in Simulation

No public EEG dataset I could find releases event sequences, the true codebook, and an explicit run-length rule together, so I checked the recovery step where the answer is known: on simulated designs. Each simulation copies the case-study design. Forty-eight codes stand for 12 words, each read by two speakers in two tokens; four words make up each of three conditions. Every simulated recording plays five blocks, each a random order of the 48 codes, under the rule that no more than three consecutive trials share a condition or a speaker, also across block borders. I drew a true codebook, generated K distinct code orders from it, and ran the recovery exactly as in Section III-C. The codebook was either *paired*, with codes $2k-1$ and $2k$ the same word and token from the two speakers, or *random*. As a negative control I generated orders with no run rule at all.

Table I gives the results. With four distinct orders, as in the case study, the speaker split was recovered uniquely and correctly in every simulation, whatever the codebook layout, and orders generated without the rule never allowed any split without a run of four. A unique speaker split with no violation is therefore strong evidence both that the rule was used and that the split is right. With only two orders the true split was always among the solutions but was unique in 55–78% of simulations, so a single order is not enough.

The condition split was never identified by the rule alone (at least 2000 solutions, the enumeration cap, in every simulation). Under the pairing assumption, the true grouping was always among the solutions when the assumption was true, but it was the only one in just 34% of simulations. When the assumption was false, the solver still returned a single, wrong grouping in 23% of simulations, and orders made without any rule did so in 16%. A unique condition grouping under an assumption is thus weak evidence on its own; it narrows the search, and the test on the EEG has to carry the weight. This is why the recovered condition labels are compared with other rule-satisfying groupings, and with groupings built from random pairings, and not only with shuffled labels.

TABLE I
Label recovery on simulated designs with known codebooks. "Unique and correct": the solver returned exactly one solution and it was the true one. Codebook layout: paired (codes $2k-1$ and $2k$ are the same word from the two speakers) or random. Pairing assumption: codes $2k-1$ and $2k$ share a condition.

	Rule applied paired	Rule applied random	No rule (control)
Four distinct orders (as in the case study), 100 simulations each			
Speaker: unique and correct	100%	100%	—
Speaker: any split without a run of four	100%	100%	0%
Condition, rule alone: identified	0%	0%	—
Condition + pairing: unique and correct	34%	0%	—
Condition + pairing: unique but wrong	0%	23%	16%
Condition + pairing: no solution	0%	28%	45%
Two distinct orders, 40 simulations each			
Speaker: unique and correct	55%	78%	—
Speaker: any split without a run of four	100%	100%	0%
Eight distinct orders, 40 simulations each			
Speaker: unique and correct	100%	100%	—
Speaker: any split without a run of four	100%	100%	0%
Condition, rule alone: solutions (median)	482	662	—
Condition + pairing: unique and correct	100%	0%	—
Condition + pairing: no solution	0%	100%	100%

More recordings with distinct orders change this. With eight orders the speaker split was again always recovered, the rule alone left a few hundred condition groupings instead of thousands, and the pairing assumption returned the true grouping in every simulation when it was true and no grouping at all when it was false. Recordings with further trial orders from the same experiment would therefore turn the condition grouping from a hypothesis into a test.

V. Case Study: Dataset

I use the Magyari canine EEG release. The raw data folder holds 17 recordings in Neuroscan CNT format. I name each dog `dog_` plus the recording number. This is my own label, not a participant list from the original authors.

Each recording should have eight channels sampled at 1000 Hz: two EEG channels (`FZ`, `Cz`), two eye channels (`F7`, `F8`), and four auxiliary channels (`X1`, `X5`, `X4`, `X6`). The file pattern, channel roles, and sampling rate are verified. Metadata completeness is unknown, and the dog numbering is inferred.

The design should have trigger codes 1–48, five repeats of each code, and 240 stimulus events per complete recording. Each sound is one of three condition types: `known`, `similar_nonsense`, or `dissimilar_nonsense`, with 80 trials each. Sounds should start 2.5–3.0 s apart, and no more than three trials in a row should share a condition or speaker.

I audited the events in all 17 recordings. Nine match the design exactly: `dog_18`, `dog_23`, `dog_26`, `dog_27`, `dog_`

37, dog_44, dog_45, dog_49, and dog_63. Eight need review: dog_20, dog_34, dog_38, dog_40, dog_48, dog_52, dog_64, and dog_65. Their problems are missing or extra repeats, recording gaps, and a few keypad or keyboard marks. Code 0 appears in dog_34, dog_40, and dog_52. It is a Neuroscan recording-control marker, not a sound (Section VI-A).

a) Why 16 dogs: I did not record any dogs myself, so the sample is fixed by what the public release contains: 17 recordings, the same number of dogs as the original study reports [1]. I exclude one, dog_40, which leaves 16 dogs and 2820 epochs. The exclusion was inherited from an earlier analysis group rather than set by a pre-written rule. The recording has 236 of the 240 expected sound events, failed an older quality check, and has the largest raw amplitude in the dataset, although that amplitude is slow drift and the recording is not an outlier after filtering: with a 1 s high-pass its largest channel standard deviation is 45.8 μV, while retained recordings reach 934 μV (dog_63). I therefore also report the main decoding analysis with all 17 dogs; the conclusion does not change (Section VII-F). The number that matters for statistics is the 16 dogs, not the 2820 epochs: epochs from the same dog are not independent, so all testing and uncertainty estimates work at the level of dogs.

A. Where the Condition Labels Come From

I keep two questions apart: are the trigger codes intact, and do I know what each code meant? Code range, event count, condition names, and randomization rules can be checked directly. That tells me whether the recordings have the expected numeric event structure.

The link from trigger code to sound is not verified. I use a candidate mapping (status unverified_candidate): the 48 audio files sorted alphabetically are assigned to codes 1–48. All 48 files exist, but having the files does not tell me which code played which sound. The labels known=1, similar_nonsense=2, and dissimilar_nonsense=3 are my own labels for epoching, not original evidence.

I did not find an original trigger codebook, a presentation or trial list with codes and file names, a task script, or per-recording stimulus logs. The recovery report says recovery_status=not_recovered, verified_mapping_available=false, and condition_level_neuroscience_claims_permitted=false. So the alphabetical labels, used in the first part of the analysis, are candidate labels. They are fine for checking coverage and pipeline behavior, but not for claims about what the dogs understood.

The candidate mapping also fails a design check. Applied to the recorded lists, it produces runs of the same condition or speaker longer than the published maximum of three. That shows this mapping cannot be the one used. Section VII-K uses the same rule, in reverse, to recover the mapping.

VI. Case Study: Implementation

A. Reading the Raw Files

The raw data are Neuroscan CNT recordings. My file reader reads the CNT header, the channel records, the continuous samples, and the event table. I read samples as 32-bit numbers. For each recording the reader checks that the data block divides exactly by the channel count and sample width, subtracts the stored baseline, and applies the stored calibration. Channel roles come from the channel names: FZ and Cz are EEG, F7 and F8 are eye (EOG), and X1, X5, X4, and X6 are auxiliary. Event rows that fall outside the data are skipped with a warning.

The key import problem was sample width. An earlier loader read the 32-bit samples as 16-bit. That split every sample in two, doubled the apparent recording length, and produced a pattern of alternating saturated and flat channels. My audit checked 32-bit divisibility, the split-sample signature, checksum agreement, and header scaling for every file. The correct settings are data_format="int32" and recompute_n_samples=True, because the header sample counts are invalid. This fixes the import. It does not show that every recording is usable for every later analysis.

Raw-data validation (131 PASS, 10 WARN) flagged eight recordings whose event counts differ from 240 and two with incomplete metadata. Event validation (134 PASS, 37 WARN) traced code 0, which appears in dog_34, dog_40, and dog_52, to a Neuroscan recording-control marker; it is now excluded from stimulus checks. The run-rule WARN fires in every recording, including the nine that match the design exactly, because the alphabetical mapping itself breaks the rule (Section VII-K).

B. Quality Control and the Final Group

QC computed duration, sampling rate, channel amplitude and variance, peak-to-peak values, finite-sample statistics, event counts, and epoch summaries against the dataset specification. Channel thresholds were left unset, so no channel was flagged (62 PASS, 7 WARN; the WARNs are event counts and the unset thresholds). An older cleaning helper had failed all 17 recordings with thresholds that were not saved, so I do not use it. The final group is 16 dogs and 2820 epochs (Section V).

C. Preprocessing, Epochs, and Features

Continuous data were band-pass filtered at 1–40 Hz (zero-phase FIR), notch filtered at 50 Hz, and resampled to 250 Hz. I applied no re-referencing and no ICA correction. Events with codes outside the 48-code mapping, including code 0, were skipped. Epochs run from -0.2 to 1.0 s around sound onset and are corrected using the -0.2 to 0 s interval as baseline. An epoch is rejected if its peak-to-peak amplitude on either EEG channel exceeds 150 μV. This threshold is fixed, not fitted per dog; eye and auxiliary channels are not used for rejection. After rejection, 103 to 227 epochs per dog remain (2820 in total).

Each epoch is summarized by 11 features per channel on all eight channels, giving 88 features: Welch band power in delta (1–4 Hz), theta (4–8), alpha (8–13), beta (13–30), and gamma (30–40 Hz); the three Hjorth parameters (activity, mobility, complexity); line length; mean; and variance. Channels X4 and X6 are empty, so 22 of the 88 features carry no signal and 10 are exactly constant. The remaining 66 features come from two

EEG, two eye, and two working auxiliary channels. Features are standardized inside each training fold before classification. A second feature set applies the same extraction to the 650–800 ms window only.

D. Decoding and Shuffled Baselines

Decoding is tested on dogs the model has not seen. In each of 16 folds, all epochs from one dog form the test set and the rest form the training set. The scaler and the classifier are fitted only on the training dogs. The main score is the macro AUC: for each of the three conditions I compute the area under the ROC curve (0.5 is chance, 1.0 is perfect) for that condition against the other two, average the three, and then average over the 16 dogs. Accuracy is secondary.

The main classifier is a standard scaler followed by multinomial logistic regression with balanced class weights, using the repository's fixed settings (`lbfgs`, `max_iter=2000`, default L2 regularization, no tuning). A fixed ridge classifier and a shallow random forest (100 trees, depth 3, minimum leaf size 5) are sensitivity checks only. The main score for the alphabetical labels is the macro AUC of logistic regression on the 16-dog group; the main test for the recovered labels is described in Section VI-G. Everything else is secondary or exploratory.

Each score is compared with a shuffled-label baseline, with $p = (1 + \#\{\text{null} \geq \text{observed}\})/(1 + N)$; lower-tail and two-sided values are also reported. The main baseline shuffles labels within each dog (1000 shuffles). I also used a code-level shuffle (1120), a circular shift of each dog's label sequence (280), a decoder on features standardized within dog (840 label and 560 code-level shuffles), and a model family of logistic regression, ridge, and random forest against 232 shared shuffles with the best of the three as the statistic. Bootstrap intervals resample dogs and are approximate, because fold models share most of their training data; the permutation p value is the main result.

I repeated the analysis on the nine exact-design recordings (1601 epochs) and with `dog_40` included (17 dogs, 3010 epochs). The 16-dog group was fixed in an earlier repository version whose pipeline already included decoding, and I have no timestamped plan, so I do not claim it was fixed before decoding was seen.

E. Could the Test See a Real Effect?

I rebuilt every epoch from the raw recordings with the same steps (same 2820 epochs and labels; macro AUC 0.5045 against 0.5034–0.5036) and added a simulated difference between `known` and `dissimilar_nonsense` words: `known` epochs received $+\alpha/2$ times the template, `dissimilar_nonsense` epochs $-\alpha/2$ times it, and `similar_nonsense` epochs nothing, so the known-versus-dissimilar difference is α times the template. The template copies the shape of the dogs' own average response to a sound, taken separately for FZ and Cz (peak 2.6 μV at FZ and 1.6 μV at Cz, near 0.36 s at FZ), and is measured in multiples of that response. At "1 times", the known-versus-dissimilar difference is as large as the response to the sound itself. I tried two feature sets: the 88 whole-epoch features, and 20 simple features (mean signal at FZ and Cz in ten 100 ms slices after the sound). The decoder is the main one: three classes, always tested on an unseen dog. A simulated effect counted as found when the score beat the 95th percentile of shuffled scores (0.517 for the 88 features, 0.520 for the 20). I repeated each size eight times on random 75% subsets of each dog's epochs. I then extended the check to effects that differ between dogs: the latency shifted by a random -150 to $+150$ ms per dog, the size multiplied by a random factor between 0 and 2 per dog, the sign reversed in a random quarter of the dogs, and all three together. Each size and variant was repeated 40 times (time-slice features) or 20 times (88 features) on random 75% subsets, and each repeat was compared with the 95th percentile of label-shuffled scores computed on subsets of the same size (400 and 60 shuffles). I give 95% Wilson intervals for the share of repeats in which the effect was found.

As a sanity check I asked whether the decoder can tell the moment before a sound (-0.2 to 0 s) from the moment just after it (0.05–0.25 s), against 100 random swaps of the two labels inside each epoch, and tested the mean signal in three windows across dogs.

F. Dog Versus Condition

Confound checks are associations, not causal tests, and the dog is always the unit. For each of the 66 working features I measured how much of its variation is explained by the dog, and, after removing dog means, by the condition (chance for condition is $2/(N - K)$, with N epochs and $K = 16$ dogs). I tested whether a model can name the dog from the features (five blocks of time held out in turn). I also decoded condition within each dog alone (five blocks of time held out in turn) against 150 label shuffles and 150 circular shifts, and tested whether the spread of per-dog scores exceeds its shuffled spread. The 96-version analysis and its family-wise test are described in Appendix A-D.

G. Recovering the Labels From the Design

The published design allows no more than three consecutive trials of the same condition or the same speaker. I took the event sequences of the nine exact-design recordings (four distinct code orders) and listed every window of four consecutive events: 948 distinct windows. I scored the alphabetical labels and 2000 random groupings of the same sizes (three conditions of 16 codes, two speakers of 24), and then solved for all groupings with no violation (Section III-C). For the condition I also tested one structural assumption, that codes $2k - 1$ and $2k$ belong to the same condition, which would hold if consecutive codes were the same word read by the two speakers. To judge how much this assumption alone narrows the solutions, I solved the same problem under 150 random pairings of odd with even codes. As a check on recordings not used for solving, I counted violations in the other eight recordings (16 further windows from uninterrupted stretches with 2.3–3.3 s spacing).

The recovered labels were fixed before any EEG was used and tested once, although the recovery procedure itself was

TABLE II
OUTCOME OF EACH VALIDATION STAGE, FROM `reports/publication/publication_evidence.json` AND `reports/validation/*.json`.

Stage	Status	PASS	WARN	FAIL
Data validation	WARN	131	10	0
Decoder validation	WARN	134	37	0
Event validation	WARN	9	9	0
Preprocessing validation	PASS	1	0	0
Epoch validation	PASS	1	0	0
QC validation	WARN	62	7	0
Confound validation	WARN	2	3	0
Statistical evaluation	WARN	0	3	0
Reproducibility validation	WARN	5	1	0

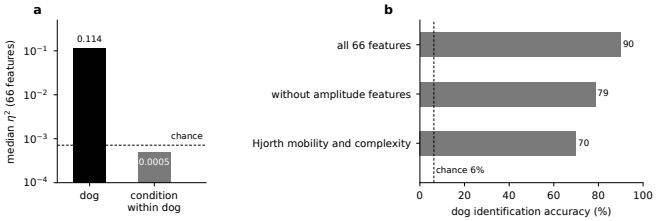


Fig. 2. (a) Median share of variation explained by dog and by condition within dog, across 66 features (log scale; dashed, chance for condition). (b) How often a model names the right dog, from three feature subsets (dashed, chance).

developed on this dataset (see Limitations). The primary test is three-class decoding of the recovered condition groups with the 20 time-slice features (chosen because the sensitivity check shows they are the more sensitive set), tested on unseen dogs, against (i) 1000 within-dog label shuffles, (ii) 800 uniformly random groupings of the 48 codes into three groups of 16, (iii) 150 other groupings that also satisfy the rule (found with randomized solver objectives), and (iv) 92 groupings that satisfy the rule under random odd-even pairings. The 88 features were tested against 300 within-dog shuffles. Follow-up tests, run after the primary one, were two-class decoding for each pair of groups, decoding in single 100 ms windows with a family-wise maximum, the late-window amplitude per group, the primary test under other preprocessing choices, and decoding of the speaker split; each is compared with its own code-level baseline, and the counts are given with the results.

H. Label-Free Trigger-Code Tests

Before recovering labels I tested, without using any labels, whether individual codes carry repeatable signal within and across dogs and whether that signal follows the alphabetical groups or the audio. The methods and results are in Appendix A-E.

VII. CASE STUDY: RESULTS

A. Overview of the Checks

Table II gives the outcome of each stage. The overall status is `NOT_READY`: the pipeline ran end to end, but problems remain, and no WARN or FAIL was turned into a PASS.

B. Recordings and Events

All 17 files were valid EEG once read as 32-bit samples and matched the official checksums. Nine recordings match the 240-event design and 2.5–3.0 s spacing exactly; eight need review (Section V). All 48 sound files were found, but none of the design records listed in Section V-A.

C. Dog Identity and Condition in the Features

On the 66 working features (Fig. 2). The dog explained a median 0.114 of the variation in a feature (middle half of features 0.018–0.244). The condition within a dog explained 0.0005, below the 0.0007 that chance alone gives; the dog explained about 230 times more (the condition value is at chance, so the ratio is only a rough guide). A model could name the dog 90% of the time (chance 6.3%), 79% without amplitude features, and 70% from just two shape measures (Hjorth mobility and complexity). Decoding condition within each dog alone was also at chance (mean 0.478 against 0.495 for shuffled labels, $p = 0.96$), and the spread between dogs matched the shuffled spread ($p = 0.39$).

D. Decoding With the Alphabetical Labels

The main model used 88 features from 2820 epochs in 16 dogs. The macro AUC was 0.503 (fold SD 0.0228; dog-bootstrap 95% CI 0.492–0.515; t interval over the 16 folds 0.491–0.516), and accuracy was 0.335 (SD 0.0311). Chance accuracy is not exactly one third because artifact rejection leaves unequal class counts within dogs (the largest gap is 17 epochs, in `dog_48`: 50, 54, and 37), so I use the shuffled-baseline mean accuracy (0.333) as the reference. Eight of the 16 held-out dogs had a macro AUC below 0.5 (range 0.474–0.547).

The repository's leakage checks record that no dog appeared in both training and test folds, the scaler was fitted inside each training fold, and there was no tuning. These checks say nothing about whether the labels are right.

E. Shuffled-Label Baselines

I recomputed the main baseline with 1000 within-dog label shuffles in a separate environment (scikit-learn 1.7.2; real macro AUC 0.5036 against the repository's 0.5034). The shuffled scores had mean 0.500 (SD 0.0109) and 95th percentile 0.517. The observed 0.504 is 0.3 standard deviations above the shuffled mean; 398 of 1000 shuffles scored at least as high, so $p = 0.40$ (lower tail 0.60; two-sided 0.80). No held-out dog reached $p < 0.05$ against its own baseline (smallest $p = 0.076$).

The code-level shuffle ($p = 0.36$), circular shift ($p = 0.35$), and a decoder with features standardized within each dog ($p = 0.33$) agree (Table III).

All baselines agree that the result is a null under the alphabetical labels. None shows that the EEG carries no information.

TABLE III
REAL MACRO AUC AGAINST SHUFFLED BASELINES (16 DOGS; 0.504 IS THE RECOMPUTED 0.5036, THE REPOSITORY VALUE IS 0.5034). $p_{\geq}$, $p_{\leq}$, AND p_2 ARE THE UPPER-TAIL, LOWER-TAIL, AND TWO-SIDED EXACT p VALUES.

Baseline (features)	N	Real	Mean (SD)	p95	$p_{\geq}$	$p_{\leq}$	p_2
Label shuffle (raw 88)	1000	0.504	0.500 (0.0109)	0.517	0.40	0.60	0.80
Code-level (raw 88)	1120	0.504	0.500 (0.0104)	0.517	0.36	0.64	0.72
Block shift (raw 88)	280	0.504	0.500 (0.0101)	0.515	0.35	0.65	0.70
Label shuffle (z-scored 66)	840	0.505	0.500 (0.0111)	0.519	0.33	0.67	0.66
Code-level (z-scored 66)	560	0.505	0.500 (0.0108)	0.517	0.29	0.71	0.59

TABLE IV
CONDITION DECODING IN OTHER GROUPS OF DOGS (TESTED ON UNSEEN DOGS; DOG-BOOTSTRAP INTERVALS; P95 FROM WITHIN-DOG LABEL SHUFFLES; 1000 SHUFFLES EXCEPT 560 FOR THE 17-DOG GROUP).

Group	Dogs	Epochs	Macro AUC	95% CI	Baseline p95
Final group	16	2820	0.503	0.492–0.515	0.517
Event-valid	9	1601	0.480	0.458–0.502	0.521
With `dog_40`	17	3010	0.500	—	0.518

F. Other Groups of Dogs and Other Models

No group of dogs beat its own baseline (Table IV). The event-valid group (nine exact-design recordings) scored 0.480, below its shuffled mean (0.4995, SD 0.0135), with a lower-tail $p = 0.070$ (two-sided 0.14). I read this as noise, since the main baseline mean is 0.500 and the other groups show no below-chance tendency. Including `dog_40` (17 dogs, 3010 epochs) gave 0.500 (baseline mean 0.500, p95 0.518, 560 shuffles; $p_{\geq} = 0.50$), and the `dog_40` fold itself scored 0.505, so the exclusion does not change the conclusion.

Logistic regression, ridge, and a shallow random forest scored 0.504, 0.501, and 0.506 against a shared 232-shuffle baseline that uses the best of the three as the statistic ($p = 0.53$).

G. Response to the Sound Itself

The average response across the 16 dogs was small (Fig. 3a): $+1.08$ μV from 0.30 to 0.60 s ($p = 0.029$ after correcting for three windows), mostly at FZ, and clear in only 4 of 16 dogs on their own. A decoder tested on unseen dogs could tell the moment before a sound from the moment after it, but only just: 0.518 to 0.526 across feature sets ($p = 0.0099$ to 0.040; Fig. 3b).

H. How Large a Known-versus-Dissimilar Difference Can Be Found?

Table V gives the answer. With the 88 whole-epoch features, a simulated difference of 4 times the average sound response was not found, 6 times was found in 38% of repeats, and 8 times in every repeat. With the simple time-slice features, 1.5 times was found every time. At the FZ peak that is about 21 μV against 4 μV. Part of the gap comes from the features: only 22 of the 88 come from the two EEG channels where the difference was added. The scores with the real labels (0.500 and 0.505) match the zero-difference row. With eight overlapping repeats these cut-offs are approximate.

TABLE V
SIMULATED KNOWN-VERSUS-DISSIMILAR DIFFERENCE IN MULTIPLES OF THE AVERAGE SOUND RESPONSE. SCORE (MEAN OF EIGHT 75% SUBSETS) AND SHARE OF REPEATS THAT BEAT THE 95TH PERCENTILE OF SHUFFLED SCORES. DASHES: NOT RUN.

	Simple time slices		88 features	
Size	Score	Found	Score	Found
0	0.498	0%	0.502	0%
0.5	0.494	0%	–	–
1	0.508	0%	–	–
1.5	0.529	100%	–	–
2	0.551	100%	0.490	0%
4	0.628	100%	0.498	0%
6	0.697	100%	0.514	38%
8	–	–	0.540	100%
12	–	–	0.609	100%
16	–	–	0.680	100%

I. Effects That Differ Between Dogs

The sensitivity shown above assumes an effect with the same shape, timing, and sign in every dog. Fig. 4 relaxes that. With the time-slice features, an effect found in every repeat at 2 times the average sound response when it was the same in every dog was found in 55% of repeats when its latency shifted by up to 150 ms between dogs (all repeats from 3 times on). Varying its size between dogs changed little. Reversing its sign in a quarter of the dogs mattered most: 53% at 3 times and 78% even at 6 and 8 times. With all three varying, 65% were found at 8 times. The 88 features needed 8 times for a fixed effect (100%; 35% at 6 times), and with the sign reversed in a quarter of the dogs only 10% were found at 8 times and all at 12 times (these 20-repeat rates differ slightly from the 8-repeat run above, e.g. 35% against 38% at 6 times). Leave-one-dog-out decoding can therefore rule out large effects shared by most dogs, and says little about effects whose sign or timing differs between dogs.

J. Other Checks

Four further checks are reported in the appendix. Feature structure changed with position in the session in all 16 dogs (Kruskal–Wallis $\epsilon^2 = 0.152$), but position was balanced across conditions and a decoder on position alone scored 0.497 (Appendix A-B). Pre-sound amplitude predicted post-sound amplitude, but no residual remained once built-in coupling and amplitude carry-over were removed ($p = 0.24$; Appendix A-C). Across 96 analysis versions the best score was not unusual once the selection was counted (family-wise $p = 0.14$), and a higher score at the 100 μV limit was borderline ($p = 0.051$) and is unresolved (Appendix A-D). Without using any labels, individual codes carried some signal shared across dogs (cross-dog agreement 0.088, Holm-corrected $p = 0.002$), which did not follow the alphabetical conditions, words, speakers, or the audio (Appendix A-E); this is why recovered labels are tested against code-level baselines.

K. Recovering the Labels From the Design

The alphabetical labels break the published rule far more often than a correct mapping could (Fig. 1a,c). Over the 948 windows of four consecutive trials in the exact-design

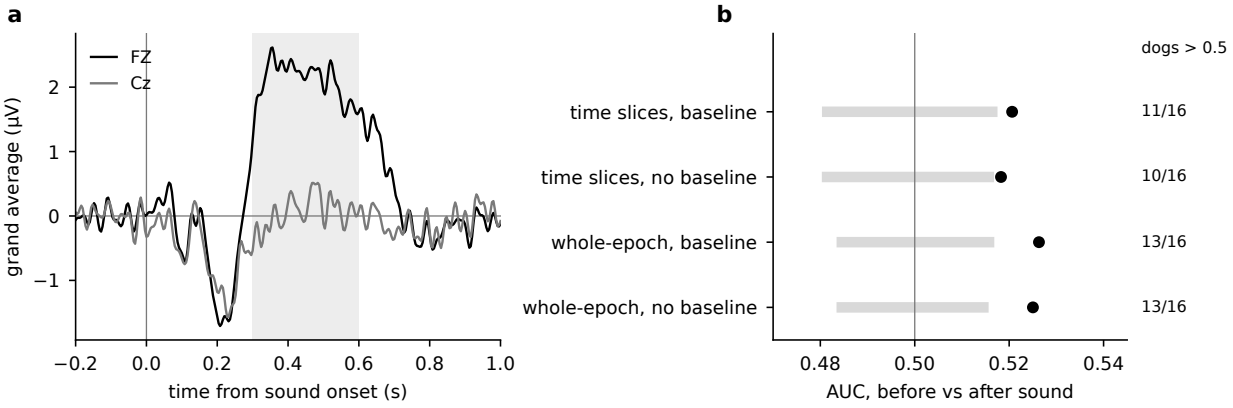


Fig. 3. Before-versus-after-sound check. (a) Average response at FZ and Cz over 16 dogs; shaded, 0.30–0.60 s. (b) Score for telling before from after the sound (points) against the 5th–95th percentile of shuffled scores (grey bars), with the number of dogs above 0.5.

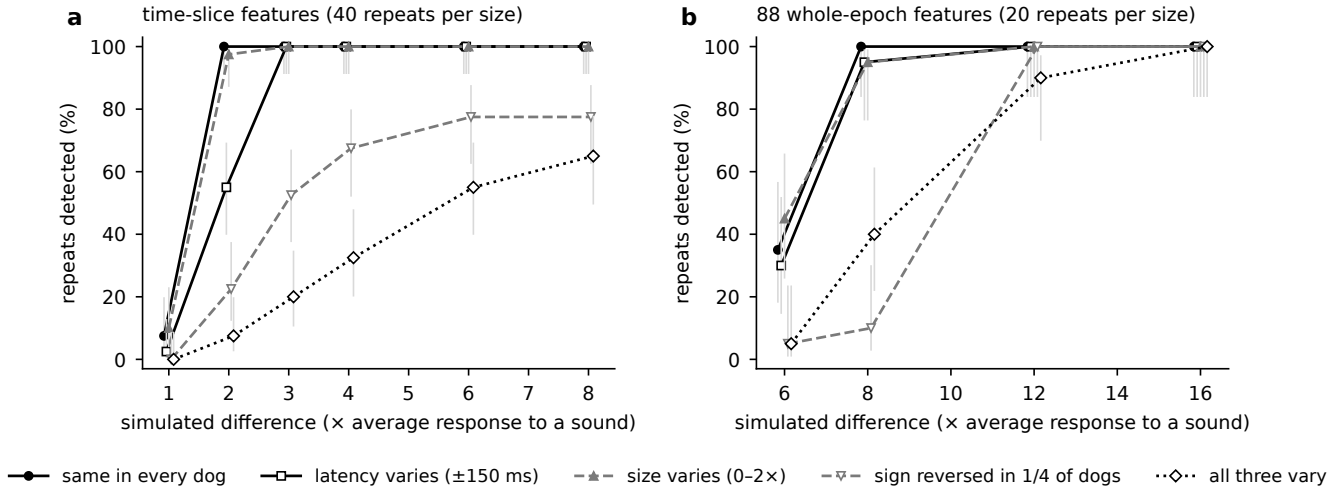


Fig. 4. Share of repeats in which a simulated known-versus-dissimilar difference was found, by size (in multiples of the average sound response) and by how it varies between dogs. Each repeat uses a random 75% of every dog's epochs and is compared with the 95th percentile of label-shuffled scores on subsets of the same size. Vertical lines: 95% Wilson intervals. Points are offset slightly along the horizontal axis for legibility.

recordings, they give 94 runs of four trials from the same speaker and 19 from the same condition. A correct mapping gives none. Random groupings of the same sizes give 104 and 27 on average (fewest in 2000 draws: 62 and 8), so the alphabetical speaker assignment is no better than chance.

Solving the rule identified the speaker split exactly. Of all ways to divide the 48 codes into two groups of 24, only one has no run of four: odd codes in one group and even codes in the other. This needed no further assumption. The split also holds in the eight recordings not used for solving, apart from one window in `dog_40`.

The condition split is not identified by the rule alone: more than 150,000 groupings into three sets of 16 satisfy it. Grouping codes in fours ($4k-3$ to $4k$) is impossible under the rule. Assuming that codes $2k-1$ and $2k$ share a condition leaves exactly one grouping: G0 = {1, 2, 9–12, 15–18, 23, 24, 27, 28, 37, 38}, G1 = {3–8, 13, 14, 21, 22, 29, 30, 35, 36, 43, 44}, and G2 = {19, 20, 25, 26, 31–34, 39–42, 45–48}. Each group holds eight odd and eight even codes, as a condition with both speakers should. This assumption is not strongly supported by the solver: half of random odd-even pairings also allow at least one grouping (75 of 150), and 23% allow exactly one. The simulations in Section IV make the same point. The condition grouping is therefore a hypothesis from the design, to be tested on the EEG; the speaker split is a result of the design. Group names are not identified.

L. Testing the Recovered Labels on the EEG

With the recovered condition groups, three-class decoding with the time-slice features on unseen dogs scored 0.537 (Fig. 1d). Within-dog label shuffles gave a 95th percentile of 0.518 ($p = 0.001$, the minimum with 1000 shuffles). The stricter code-level baselines gave: random groupings of the codes, mean 0.505 and 95th percentile 0.524 ($p = 0.005$, 800 groupings); other groupings that satisfy the rule, mean 0.509 and 95th percentile 0.526 ($p = 0.013$, 150); and 92 rule-satisfying groupings from the feasible random odd-even pairings, mean 0.509 and 95th percentile 0.526 ($p = 0.011$).

The alphabetical labels scored 0.500 with the same features. Fifteen of the 16 held-out dogs scored above 0.5 (median 0.531). The 88 whole-epoch features scored 0.506 ($p = 0.29$ against 300 within-dog shuffles), as expected from their lower sensitivity (Table V).

The result held under other analysis choices, each against its own 300 random groupings: 0.535 with `dog_40` included ($p = 0.003$), 0.533 with a 100 μV limit ($p = 0.027$), 0.535 with a 200 μV limit ($p = 0.003$), 0.536 without baseline correction ($p = 0.003$), and 0.537 with a 10 Hz low-pass ($p = 0.007$).

Follow-up tests show where the signal sits (Fig. 5). Groups G0 and G1 could not be told apart (two-class AUC 0.503, $p = 0.53$), while G2 differed from both (0.551 and 0.573, $p = 0.0025$ each, the minimum with 400 splits). Over ten 100 ms windows, the 200–300 ms window scored 0.531 (family-wise $p = 0.003$ against 300 groupings); 300–400 ms was nominally above its baseline (0.525, $p = 0.013$; family-wise 0.07). At 0.65–0.80 s, G2 was 0.68 and 0.87 μV more negative than G0 and G1, which was not significant ($p = 0.26$ and 0.21; 12 and 10 of 16 dogs). The recovered speaker split was decoded weakly (two-class 0.528 against a 95th percentile of 0.532 for 500 random 24/24 splits, $p = 0.09$).

This pattern resembles the original report, in which known words differed from dissimilar nonsense words at 200–300 ms (with one of two cleaning methods) and at 650–800 ms, while known words and similar nonsense words did not differ [1]. If G2 were the dissimilar nonsense words, the recovered labels would reproduce that pattern. That naming is an inference from the EEG and is not evidence for the mapping in its own right. The label-free code tests of Section A-E add little: codes in the same recovered group were only slightly more alike than codes in different groups ($p = 0.11$; speaker $p = 0.13$), and codes $2k - 1$ and $2k$ were not more alike than random odd-even pairs ($p = 0.57$).

M. Reproducibility

The archived run recorded checksums for 17 raw files and 48 derived files and reports, software versions (including `mne=1.6.1` [29]), and random seed 42. Its single WARN is that the code had uncommitted changes (`git_dirty=True`). All other numbers come from separate scripts run on the saved features or on epochs rebuilt from the raw recordings (Python 3.10, scikit-learn [30] 1.7.2), so they are tied to the repository's outputs but not to its recorded commit.

VIII. Discussion

A. What the Pipeline Changed in This Case

An accuracy-first analysis would have reported one number, 0.503, and read it as an absence of word-condition information. The pipeline showed instead that the recordings needed a 32-bit import and event review, that the decoder could only have found large effects, and, most importantly, that the labels are inconsistent with the published design: their speaker assignment breaks the rule as often as a random one does.

B. What Label Recovery Can and Cannot Do

The recovery stage turned a published sentence about randomization into a test of the labels, and in part into the labels themselves. With four trial orders the rule identified the speaker split exactly, which the simulations show to be reliable. It did not identify the condition split; the assumption that narrowed it to one grouping is only weakly supported by the solver with four orders, though eight orders would settle it (Section IV). The recovered condition labels are therefore a hypothesis generated from the design, and the EEG test is what supports them.

That test was built to be hard to pass. The labels were fixed before the EEG was used, and the main comparison was not a within-dog shuffle but other groupings of the same codes. This matters: in these data any grouping of codes decodes slightly above shuffled labels (mean 0.505 against 0.500), because individual codes carry some signal of their own (Section A-E). A within-dog baseline alone would have overstated the evidence. Against random groupings, against other rule-satisfying groupings, and against groupings built from random pairings, the recovered labels still scored higher, under every preprocessing choice I tried, and in 15 of 16 held-out dogs. The effect also had a specific structure: one group differed from the other two, at 200–300 ms, which resembles the published finding. None of this makes the mapping verified. The pairing assumption could be wrong in a way that still groups codes by some property of the sounds, and the effect is small (0.537). Confirmation has to come from the data providers, and the recovery makes that request concrete: three yes-or-no questions (is the speaker given by code parity, are codes $2k - 1$ and $2k$ the same word, and is G2 the dissimilar nonsense words) replace a request for a lost codebook.

C. Sensitivity and Effects That Differ Between Subjects

The sensitivity stage explains why the 88 whole-epoch features found nothing even with the recovered labels: they needed a difference of about eight times the average sound response, against about 1.5 to 2 times for the simple time-slice features, and only 22 of the 88 features come from the two EEG channels. The 0.537 found with the recovered labels lies between the scores produced by simulated effects of one and two times the average response. The variant simulations add a general point. Leave-one-subject-out decoding finds effects that are shared by most subjects; when the sign of an effect differs in a quarter of the subjects, even large effects are often missed. A negative result from this kind of test should be reported as "no effect shared across subjects larger than a stated size" rather than "no effect".

D. Dog Identity and Session Order

Both dog identity and session order are present in the features. Identity explains about 230 times more variation than condition does within a dog, and distance to the reference cluster fell with trial position in all 16 dogs. I cannot say whether the trend reflects habituation, fatigue, arousal, electrode drift,

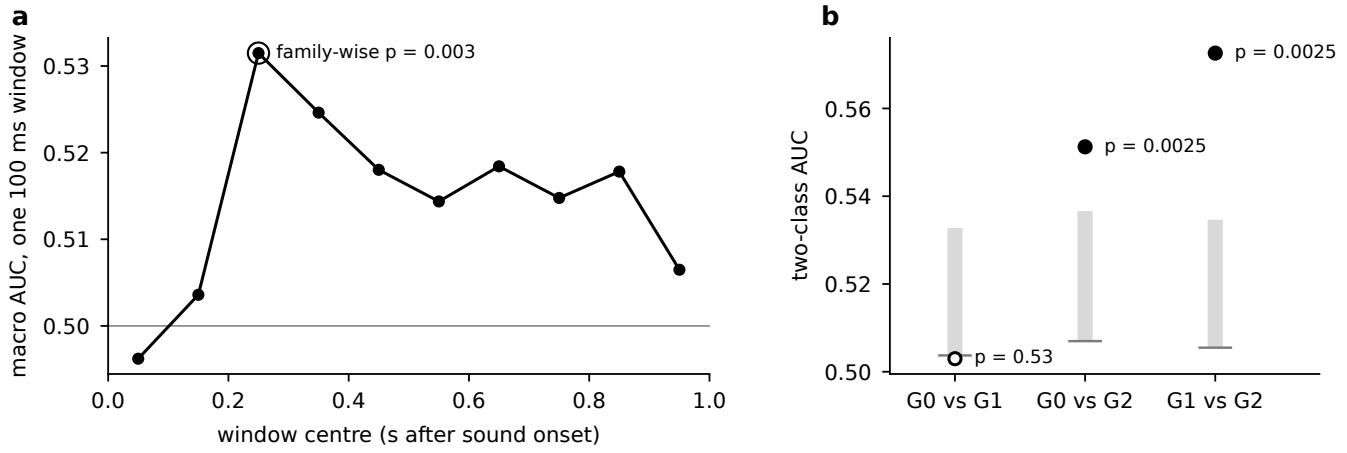


Fig. 5. Where the recovered-label effect sits. (a) Three-class decoding (unseen dogs) in single 100 ms windows; circled, family-wise $p < 0.05$ against 300 random code groupings. (b) Two-class decoding for each pair of recovered groups (points; filled, $p < 0.05$) against 400 random splits of the same codes (grey bars: mean, marked, to 95th percentile).

or something else. A classifier on these features could partly learn position in the session, which is one more reason to compare every score with baselines that keep the structure of the data.

E. What Transfers

What transfers to other datasets is the structure: a dataset specification, event and label provenance, recovery of labels from design rules when the codebook is missing, quality evidence, confound checks, testing on unseen subjects against baselines that keep the structure of the data, calibration of what the test could have found, and claim audits. The recovery step needs only event sequences and a written rule, which many published designs state. The next steps are to run the pipeline on a dataset whose codebook is known but withheld from the solver, and on an independently documented canine EEG dataset.

IX. Limitations

The main limit is still the mapping. The alphabetical labels are shown to be inconsistent with the published rule, but the recovered labels are not verified. The speaker split is identified by the design; the condition split rests on one structural assumption that the solver supports only weakly, its group names are inferred from the EEG, and the evidence for it is one test on 16 dogs with a small effect (0.537). The recovery was developed on this dataset, the follow-up tests (pairs, time windows, robustness) were chosen after the main test, and only the main test was fixed in advance. Simulation checks the solver on designs like this one, not on real recordings with a known codebook; I found no public dataset that releases event sequences, codebook, and an explicit run rule together.

Sixteen dogs limit power however the epochs are handled. All validation stages remain at WARN, the outputs came from code with uncommitted changes, two repository reports of the main score differ slightly (0.5034 and 0.5025) for reasons I did not trace, and some repository baselines rest on only 20 shuffles.

The exclusion of `dog_40` was inherited, not derived from a rule; including it did not change the results, and dropping the noisier retained recordings was not tested. No analysis plan was timestamped before decoding, and everything beyond the declared main scores is exploratory. QC thresholds describe this dataset only, confound checks are associations, and only 22 of the 88 whole-epoch features come from EEG channels.

The sensitivity unit is the peak of a marginal grand-average response (4 of 16 dogs) and is probably somewhat too large; the simulated effects vary between dogs only in the ways I chose; and the first, eight-repeat check gives approximate cut-offs. The 96-version analysis leaves a borderline pattern across artifact limits unresolved (Appendix A-D). The label-free code tests are low-powered, use only FZ and Cz, and cannot rule out a trigger-line artifact (Appendix A-E). Finally, the pipeline has been run on one dataset. A synthetic test shows the validators do not depend on constants specific to it, but that is software evidence, not biological validation.

X. Conclusion

A decoding score from a small EEG dataset means little until the data, the labels, and the test's own sensitivity have been checked. The pipeline described here does that and adds two stages: recovering labels from the experimental design when the codebook is missing, and calibrating what the test could have found. In a public canine EEG dataset, it showed that the labels used so far break the published design rule, recovered the speaker split exactly, and produced a design-consistent condition grouping that decodes above baselines built from other groupings of the same codes, with a pattern resembling the original report. The recovered condition labels remain a hypothesis until the data providers confirm them, and the case study makes no claim about what dogs understand. The method needs only event sequences and a written randomization rule, so it can be applied to other shared datasets whose codebooks were lost.

Data and Code Availability

The full pipeline, including every validator, report generator, and the reproducibility tooling described here, is licensed under the MIT license and kept in a private repository. Access is granted on request to the author (bhoomikahegde85@gmail.com). The original Magyari data and stimulus material are available through the figshare DOIs cited in the original article. The scripts for label recovery, its simulation check, and all analyses reported here are included with the code. No second-dataset results are reported here.

Appendix A
Supplementary Analyses

This appendix holds checks that support the main text but are not needed to follow it.

A. Methods for the Supplementary Checks

a) Session position (methods).: Within each dog, epoch index was scaled to 0–1. I clustered the 88-feature matrix with k-means ($k = 3$, 50 starts, seed 42). Because cluster numbers have no order, I compared position across clusters with a Kruskal–Wallis test and a within-dog shuffle of cluster labels (500 repeats). I also computed per-dog Spearman correlations between position and distance to the center of the largest cluster, with a dog-bootstrap interval. To see whether position could carry label information, I ran the same decoder on position alone (position and its square) against a 400-shuffle baseline.

b) Readiness and amplitude carry-over.: Using ridge regression tested on held-out dogs, I predicted post-sound amplitude (0.05–0.8 s) from pre-sound features (-0.2 to 0 s: mean, standard deviation, range, RMS, line length, slope, and the `FZ–Cz` correlation). The original target subtracts the pre-sound mean, which is also a predictor, so part of the association is built in. I therefore also used a decoupled target, the mean absolute deviation of the post window about its own mean, which uses no pre-sound value. As a control for carry-over I predicted the same target from the first half of the baseline only, and then tested whether the full pre-sound features predict what the control cannot (the residual), against 1000 within-dog shuffles.

c) 96 analysis versions (methods).: Finally, in the spirit of a specification curve [31], I reran the analysis under 96 different choices: artifact limit 100, 150, 200 μV or none; baseline correction on or off; low-pass at 40, 20, or 10 Hz; both feature sets; and `dog_40` excluded or included. The best version received its own baseline (60 shuffles). To correct for picking the best of the versions, I reran the 24 versions of the 88-feature analysis without `dog_40` (four artifact limits, baseline on or off, low-pass off, 20, or 10 Hz) on 196 shuffles in which the labels were shuffled within each dog once and every version read the same shuffled labels, and compared the highest real score with the highest score in each shuffle. I also compared the mean score at the 100 μV limit with the looser limits in the same way.

d) Label-free trigger-code tests (methods).: The condition labels come from matching the 48 trigger codes to the 48 audio files in alphabetical order (12 words with 4 recordings each; codes 1–22 and 23–44 from two speakers). Nobody documented this match, and a negative result only means something if the match is right. So I looked for structure in the codes without using the labels at all. I used the mean signal at FZ and Cz in 25 slices of 40 ms over the first second (standardized within dog). Each code appears once in each of five shuffled blocks, about 3.7 usable epochs per code and dog.

- *Same dog, same code (A1).* Are two epochs with the same code more alike than two epochs with different codes? The baseline shuffles codes among epochs of the same repeat.
- *Different dogs, same code (A2).* I split the dogs into two halves, averaged each code within each half, and asked whether the same code looks alike across halves (200 random splits, baseline from shuffling code labels, 10000 times). Several dogs heard the same order of codes, so a code could just reflect when in the session it was played. To guard against that, I (i) put dogs with the same order in the same half (11 order groups from the full code sequence of each recording), (ii) removed the slow drift over the session, (iii) compared only dogs with different orders, and (iv) checked by channel, time window, early versus late repeats, first versus last repeat, and 40 ms time slice (against a family-wise maximum baseline, 1000 shuffles). I also tested whether the pattern depends on the code number or its six binary digits.
- *Does the pattern match the assumed groups?* I checked whether codes in the same assumed condition, word (12 groups), or speaker file look more alike than codes in different groups.
- *Does it match the audio? (B0–B3)* Under the assumed match, I checked (B0) whether tokens of the same word look alike; (B1) whether the brain signal follows the loudness curve of the audio (Hilbert envelope, $<$10 Hz, lags 0–800 ms, 2000 random file shuffles as the comparison); (B2) whether any of six simple sound measures matches any of the 50 brain features (300 tests, corrected for the maximum); and (B3) whether a file-to-code match learned on half of the dogs (Hungarian assignment) agrees with the assumed match and predicts the other half (60 random half-splits).

I corrected the 23 tests reported in the results for multiple testing with the Holm method. To see how small a signal each test can find, I added simulated signals of known size and counted how often each test found them (eight repeats per size). A1 got a random pattern shared across dogs with per-bin spread equal to a multiple of the peak response (2.62 μV). B1 got the delayed audio envelope scaled by the same peak. A2 has no such check, because A2 is already positive without any added signal.

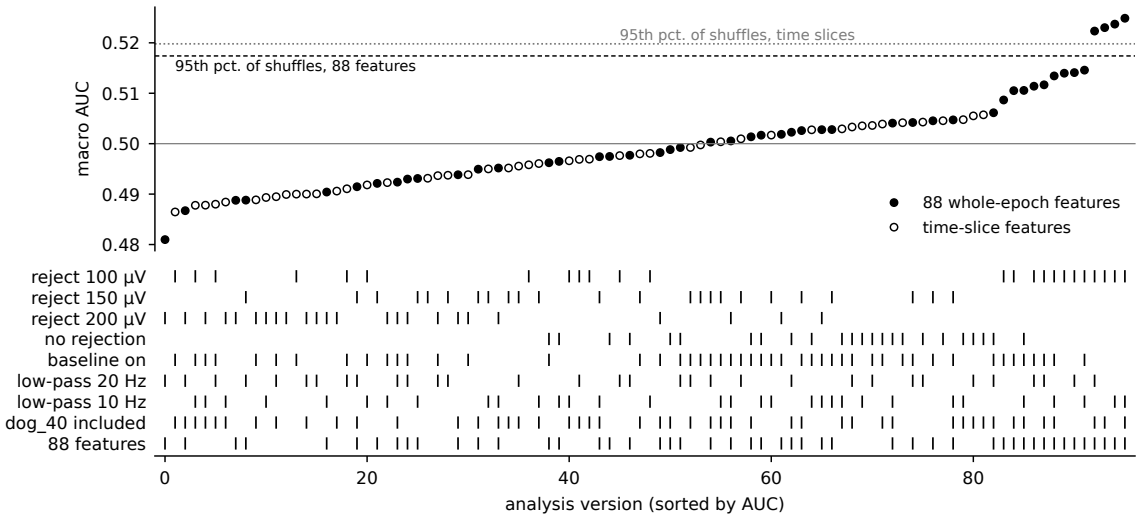


Fig. 6. Scores for all 96 analysis versions, sorted (top), with the choices behind each (bottom). Dashed: 95th percentile of shuffled scores for each feature set.

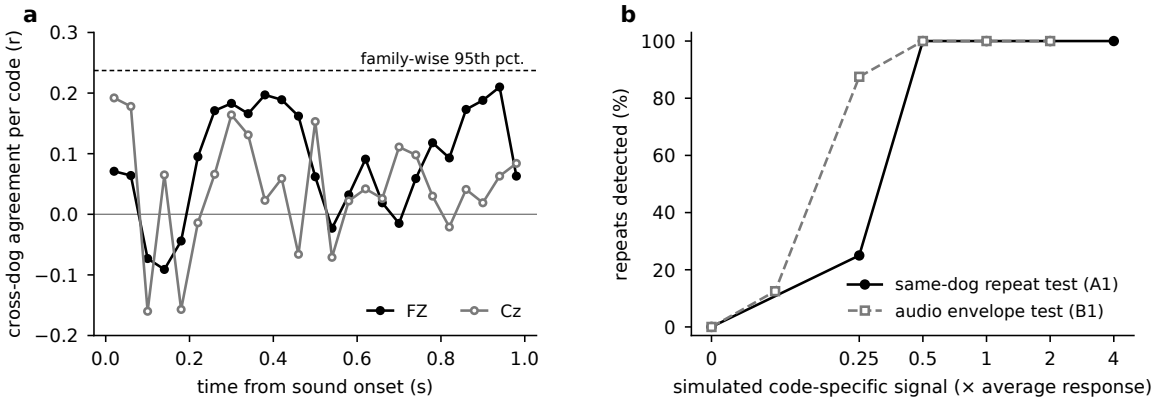


Fig. 7. Trigger-code tests. (a) How well each of the 48 codes agrees between two halves of the dogs that heard different code orders, per 40 ms slice at FZ and Cz; dashed, strict 95% cut-off over all slices. (b) Share of repeats that found a simulated code-specific signal, against its size in multiples of the average sound response, for the same-dog test (A1) and the audio-loudness test (B1).

B. Session Position

A 10-component PCA explained only 0.652 of the feature variance, so clustering used all 88 features. The three clusters had 297, 397, and 2126 epochs. Position within the session differed across clusters (Kruskal–Wallis $\epsilon^2 = 0.152$; mean position 0.38, 0.25, and 0.56 in the 297-, 397-, and 2126-epoch clusters). Shuffling cluster labels within dogs gave a baseline of about 0 (95th percentile 0.0013; $p = 0.002$, the minimum with 500 repeats), and the per-dog test was significant at $p < 0.05$ in 14 of 16 dogs (uncorrected). Distance to the center of the largest cluster fell with position (per-dog Spearman mean -0.239, SD 0.149; dog-bootstrap CI -0.310 to -0.168), and the correlation was negative in all 16 dogs. Position does not carry label information: mean position was 0.50 for all three conditions (Kruskal–Wallis $p = 0.94$), and a decoder using position alone scored 0.497 against a baseline mean of 0.499 ($p_{\geq} = 0.59$). Structure linked to position therefore cannot by itself produce above-chance condition decoding, because the labels are balanced across the session. Epoch index after rejection is a proxy for, not a measure of, elapsed time.

C. Readiness and Amplitude Carry-Over

With the original target, pre-sound features predicted post-sound amplitude strongly (Spearman $\rho = 0.716$, $R^2 = 0.570$; fold-mean $\rho = 0.715$, CI 0.695–0.737), and the control that used only the first half of the baseline gave $\rho = 0.640$ ($R^2 = 0.488$). Removing the part predictable from carry-over left a small residual ($\rho = 0.146$, $R^2 = 0.0114$; fold-mean 0.160, CI 0.115–0.205) above its baseline (p95 0.039, $p = 0.001$). I reproduced these values, but the target subtracts the pre-sound mean while the predictors include it, so part of the association is built in.

With the decoupled target the association is much weaker (Table VI). The full pre-sound model gave $\rho = 0.433$ ($R^2 = 0.155$; fold-mean 0.408, CI 0.313–0.507), still far above its baseline (p95 0.038, $p = 0.001$). The carry-over

TABLE VI
PREDICTING POST-SOUND AMPLITUDE FROM PRE-SOUND FEATURES (TESTED ON HELD-OUT DOGS; SPEARMAN ρ BETWEEN PREDICTED AND OBSERVED). BASELINES: 1000 WITHIN-DOG SHUFFLES.

Analysis	Original target	Decoupled target
Full pre-sound features	0.716	0.433
First-half baseline control	0.640	0.380
Residual after control	0.146	0.016
Residual p95 / exact p	0.039 / 0.001	0.036 / 0.24

control gave $\rho = 0.380$ ($R^2 = 0.124$), and the residual after removing it was $\rho = 0.016$ ($R^2 = -0.019$; p95 0.036, $p = 0.24$). Once the built-in coupling is removed, the pre-sound association is largely explained by amplitude carry-over, and I detect no residual readiness effect. This does not show that there is no readiness effect: the residual is small, I computed no equivalence bound, and the control uses only the earlier half of the baseline. The repository's label `exploratory_positive` for this residual is therefore not supported.

D. Robustness to 96 Analysis Choices

Across the 96 versions of the analysis (Fig. 6), the simple features scored 0.486 to 0.506 (mean 0.496), and none beat its 95th-percentile cut-off. The 88 features scored 0.481 to 0.525 (mean 0.502). Each version is compared with a fixed cut-off taken from the main version (0.517 and 0.520), not with a baseline of its own. Four versions beat 0.517, all with the 100 μV artifact limit and no baseline correction, which keeps fewer epochs (2075–2245). There is a pattern behind this: with the 88 features, the 100 μV limit scored highest (mean 0.520 without and 0.512 with baseline correction), against 0.493 and 0.503 at 150 μV and 0.489 and 0.498 at 200 μV. I do not know whether stricter cleaning exposes a small real signal or whether the cut-off simply does not fit these smaller sets of epochs. The best version tested on its own (0.524, 100 μV, no baseline correction, 10 Hz low-pass, `dog_40` excluded) had a shuffled mean of 0.499 and spread 0.013, giving $p = 0.033$ from 60 shuffles, before accounting for picking it out of many; the highest of those 60 shuffled scores was already 0.527. The best version overall (0.525) is the same setting with `dog_40` included.

To account for picking the best version, I repeated the 24 versions of the 88-feature analysis that exclude `dog_40` (four artifact limits, baseline on or off, three low-pass settings) on 196 shuffles. In each shuffle the labels were shuffled within each dog once and every version read the same shuffled labels, and I kept the highest score. The highest real score (0.524) was not unusual: the highest score in a shuffle averaged 0.515 (95th percentile 0.530), so the family-wise $p = 0.14$. The gap between the 100 μV limit and the looser limits (0.019 higher in mean score) was borderline against the same gap in shuffles ($p = 0.051$). I leave this gap unresolved.

E. Label-Free Trigger-Code Tests

a) Same dog.: Epochs with the same code were no more alike than epochs with different codes (A1 $= -0.002$; shuffled mean -0.003; $p = 0.42$; 7 of 16 dogs positive). The test found a simulated code pattern in 25% of repeats at 0.25 times the average sound response and in every repeat from 0.5 times (Fig. 7b). So it would have seen a repeatable pattern of about half the sound response, but says little about smaller ones. The p values below are not corrected. I then corrected all 23 tests in this section for multiple testing (Holm) and say below which survive.

b) Different dogs.: Averaged codes agreed between dog halves (A2 $= 0.088$, shuffled spread 0.024, $p = 0.0001$, the smallest possible with 10000 shuffles). This held when dogs with the same code order were kept in the same half (0.070, $p = 0.001$), after removing session drift (0.096, $p = 0.0001$), and with both (0.081, $p = 0.0002$). It also held over the 111 pairs of dogs with different orders (0.0086, $p = 0.023$; this number is on a different scale because each side is one dog). I cannot rule out that session position causes it: across dogs with different orders, the average position of each code still correlates at 0.058 (spread 0.18, up to 0.45). The pattern was clearer at FZ (0.089, $p = 0.004$) than Cz (0.039, $p = 0.048$). It held between early repeats (0–2) and late repeats (3–4) (0.046, $p = 0.017$) but not between only the first and last repeat (0.003, $p = 0.45$; one epoch per code and dog, so low power). By time window it was nominal only in the last (0.6–1.0 s, $p = 0.016$; earlier windows $p = 0.091$ and 0.066). No single 40 ms slice beat the strict family-wise cut-off (highest 0.21 at FZ, 0.94 s; cut-off 0.237; Fig. 7a), although FZ stayed around 0.16–0.20 from 0.26 to 0.46 s. The pattern did not depend on the code number (not significant: $p = 0.24$; binary digits $p = 0.19$), though this does not rule out a trigger artifact given only 48 codes.

After Holm correction across all 23 trigger-code tests, the four main cross-dog tests still hold (any split, position removed, playlist-disjoint halves, and both together; corrected $p = 0.002$, 0.002, 0.020, 0.004). The tests that are most careful about session position do not: the 111 different-playlist pairs (corrected $p = 0.34$; 0.22 with position removed), FZ alone (0.068), and all channel, window, and repeat splits (0.27 or higher). So the cross-dog pattern is robust to splitting by playlist, but I cannot show it for pairs of dogs with different orders once the number of tests is counted.

c) Does it match the assumed labels or the audio?: No. Codes in the same assumed condition were no more alike than codes in different conditions (difference -0.004, $p = 0.62$). The same held for words (0.006, $p = 0.37$) and speakers (-0.008, $p = 0.83$; first 22 codes against next 22, $p = 0.85$). Tokens of the same word were not more alike for the same speaker (-0.053, $p = 0.76$) or the other speaker (-0.055, $p = 0.84$), but this test has only 24 pairs and is too noisy to say much. The brain signal did not follow the loudness curve of the audio (B1 $= 0.093$ at a lag of 604 ms, shuffled mean 0.070, 95th percentile 0.106, $p = 0.13$; lags 0–800 ms; with lags of 0–300 ms the result is the same, $p = 0.50$); none of the 300 sound-measure comparisons was significant (strongest $|r| = 0.394$, $p = 0.46$ after correction); and a file-to-code match learned on half of the dogs did not recover the assumed match (1.5 of 48 codes agreed on average, against 1.0 expected by chance) or predict the other half better than

random matches ($|r| = 0.204$ against 0.242). The loudness test finds a simulated response of 0.25 times the average sound response in 88% of repeats and of 0.5 times or more in all (Fig. 7b). The six sound measures test can only call a correlation above 0.49 significant after correction for 300 tests, so it is weak.

ACKNOWLEDGMENT

I thank Lilla Magyari, Zsófia Huszár, Anna Turzó, and Attila Andics for publicly releasing the canine EEG recordings and stimulus material analyzed in this case study [1]. This research received no external funding.

AI DISCLOSURE

Generative AI tools were used as an aid in preparing this work. I take full responsibility.

REFERENCES

[1] L. Magyari, Z. Huszár, A. Turzó, and A. Andics, "Event-related potentials reveal limited readiness to access phonetic details during word processing in dogs," *Royal Society Open Science*, vol. 7, p. 200851, 2020.

[2] K. S. Button, J. P. A. Ioannidis, C. Mokrysz, B. A. Nosek, J. Flint, E. S. J. Robinson, and M. R. Munafò, "Power failure: why small sample size undermines the reliability of neuroscience," *Nature Reviews Neuroscience*, vol. 14, no. 5, pp. 365–376, 2013.

[3] J. P. A. Ioannidis, "Why most published research findings are false," *PLOS Medicine*, vol. 2, no. 8, p. e124, 2005.

[4] A. Kis, S. Szakadát, E. Kovács, M. Gácsi, P. Simor, F. Gombos, J. Topál, Á. Miklósi, and R. Bódizs, "Development of a non-invasive polysomnography technique for dogs (Canis familiaris)," *Physiology & Behavior*, vol. 130, pp. 149–156, 2014.

[5] H. Törnqvist, M. V. Kujala, S. Somppi, L. Hänninen, M. Pastell, C. M. Krause, J. Kujala, and O. Vainio, "Visual event-related potentials of dogs: a non-invasive electroencephalography study," *Animal Cognition*, vol. 16, no. 6, pp. 973–982, 2013.

[6] M. V. Kujala, J.-P. Kauppi, H. Törnqvist, L. Helle, O. Vainio, J. Kujala, and L. Parkkonen, "Time-resolved classification of dog brain signals reveals early processing of faces, species and emotion," *Scientific Reports*, vol. 10, p. 19846, 2020.

[7] ——, "Dog visual perception of faces, objects and phase-scrambled images, measured with non-invasive electroencephalography," 2020. [Online]. Available: https://doi.org/10.5281/zenodo.4114599

[8] A. Andics, A. Gábor, M. Gácsi, T. Faragó, D. Szabó, and Á. Miklósi, "Neural mechanisms for lexical processing in dogs," *Science*, vol. 353, no. 6303, pp. 1030–1032, 2016.

[9] G. S. Berns, A. M. Brooks, and M. Spivak, "Functional MRI in awake unrestrained dogs," *PLOS ONE*, vol. 7, no. 5, p. e38027, 2012.

[10] N. Kriegeskorte, W. K. Simmons, P. S. F. Bellgowan, and C. I. Baker, "Circular analysis in systems neuroscience: the dangers of double dipping," *Nature Neuroscience*, vol. 12, no. 5, pp. 535–540, 2009.

[11] F. Del Pup, A. Zanola, L. F. Tshimanga, A. Bertoldo, L. Finos, and M. Atzori, "The role of data partitioning on the performance of EEG-based deep learning models in supervised cross-subject analysis: A preliminary study," *Computers in Biology and Medicine*, vol. 196, p. 110608, 2025.

[12] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," *GigaScience*, vol. 6, no. 5, p. gix019, 2017.

[13] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, 2023.

[14] G. Brookshire *et al.*, "Data leakage in deep learning studies of translational EEG," *Frontiers in Neuroscience*, vol. 18, p. 1373515, 2024.

[15] S. E. Lazic, "The problem of pseudoreplication in neuroscientific studies: is it affecting your analysis?" *BMC Neuroscience*, vol. 11, p. 5, 2010.

[16] E. Aarts, M. Verhage, J. V. Veenvliet, C. V. Dolan, and S. van der Sluis, "A solution to dependency: using multilevel analysis to accommodate nested data," *Nature Neuroscience*, vol. 17, pp. 491–496, 2014.

[17] T. E. Nichols and A. P. Holmes, "Nonparametric permutation tests for functional neuroimaging: a primer with examples," *Human Brain Mapping*, vol. 15, no. 1, pp. 1–25, 2002.

[18] A. M. Winkler, G. R. Ridgway, M. A. Webster, S. M. Smith, and T. E. Nichols, "Permutation inference for the general linear model," *NeuroImage*, vol. 92, pp. 381–397, 2014.

[19] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. New York, NY, USA: Chapman and Hall, 1993.

[20] G. Varoquaux, "Cross-validation failure: small sample sizes lead to large error bars," *NeuroImage*, vol. 180, pp. 68–77, 2018.

[21] G. Varoquaux, P. R. Raamana, D. A. Engemann, A. Hoyos-Idrobo, Y. Schwartz, and B. Thirion, "Assessing and tuning brain decoders: cross-validation, caveats, and guidelines," *NeuroImage*, vol. 145, pp. 166–179, 2017.

[22] E. Combrisson and K. Jerbi, "Exceeding chance level by chance: The caveat of theoretical chance levels in brain signal classification and statistical assessment of decoding accuracy," *Journal of Neuroscience Methods*, vol. 250, pp. 126–136, 2015.

[23] H. Nolan, R. Whelan, and R. B. Reilly, "FASTER: Fully automated statistical thresholding for EEG artifact rejection," *Journal of Neuroscience Methods*, vol. 192, no. 1, pp. 152–162, 2010.

[24] M. Jas, D. A. Engemann, Y. Bekhti, F. Raimondo, and A. Gramfort, "Autoreject: Automated artifact rejection for MEG and EEG data," *NeuroImage*, vol. 159, pp. 417–429, 2017.

[25] M. D. Wilkinson, M. Dumontier, I. J. Aalbersberg *et al.*, "The FAIR guiding principles for scientific data management and stewardship," *Scientific Data*, vol. 3, p. 160018, 2016.

[26] K. J. Gorgolewski, T. Auer, V. D. Calhoun *et al.*, "The brain imaging data structure, a format for organizing and describing outputs of neuroimaging experiments," *Scientific Data*, vol. 3, p. 160044, 2016.

[27] C. R. Pernet, S. Appelhoff, K. J. Gorgolewski, G. Flandin, C. Phillips, A. Delorme, and R. Oostenveld, "EEG-BIDS, an extension to the brain imaging data structure for electroencephalography," *Scientific Data*, vol. 6, p. 103, 2019.

[28] L. Perron and F. Didier, "CP-SAT, part of OR-Tools," Google, https://developers.google.com/optimization/, version 9.15.

[29] A. Gramfort, M. Luessi, E. Larson, D. A. Engemann, D. Strohmeier, C. Brodbeck, R. Goj, M. Jas, T. Brooks, L. Parkkonen, and M. Hämäläinen, "MEG and EEG data analysis with MNE-Python," *Frontiers in Neuroscience*, vol. 7, p. 267, 2013.

[30] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[31] U. Simonsohn, J. P. Simmons, and L. D. Nelson, "Specification curve analysis," *Nature Human Behaviour*, vol. 4, pp. 1208–1214, 2020.